Gaussian Bandit Assignment

Starting application: https://carousel-mvp.streamlit.app/

Task: extend the Streamlit frontend with Thompson sampling for continuous Gaussian rewards with known noise standard deviation. Work with the contents of **app/front/**.

1. Create your own frontend repository

Clone the starting repository: https://github.com/hovhannisyan91/DS223_Group_Project. Copy the source contents of **app/front/** into the root of a new GitHub repository. Include the Python files, pages, **pyproject.toml**, and **uv.lock**; exclude local environments and generated caches. Credit the starting repository in your README.

Use Python 3.12 and uv. From your new project folder, run:

```
uv sync --locked
uv run streamlit run main.py
```

2. Understand the original algorithm

Use the starting application to create a carousel, submit click/no-click feedback, and inspect Analytics. Then read the corresponding frontend code. In a short README explanation, describe alternative selection, reward recording, Beta posterior updates, and how session state connects the pages.

3. Implement Gaussian Thompson sampling

Use three alternatives. Selecting alternative i produces one continuous reward:

$R \mid A = i \sim \text{Normal}(\mu_i, \sigma^2)$

The shared noise standard deviation **$\sigma > 0$ is known**; reward precision is **$1/\sigma^2$**. Each true mean μ_i is unknown to the learner. Use the same **Normal(0, 25)** prior for every unknown mean: prior mean 0 and variance 25.

Each round: sample a possible mean from each current Normal posterior; select the largest; generate one Gaussian reward; then update only the selected alternative's posterior using the reward and known σ.

True means are used by the simulator to generate rewards and by you to evaluate learning. Selection must use the learned posteriors. Keep rewards continuous, including negative values.

4. Add the Gaussian Bandit page

Register a new page in the existing navigation and keep the original pages available. Add inputs for **three true means, known σ, number of rounds, and random seed**, plus a Run button. Validate that σ is positive and the round count is a positive integer.

Display each alternative's **true mean, posterior estimated mean, and selection count**; the **total observed reward**; and the **final recommendation**, defined as the alternative with the largest posterior mean.

5. Run three simple experiments

Use the following settings and record the seed. Keep the same prior and seed across the three runs.

Experiment	True means	Known σ	Rounds
Short run	[1, 2, 3]	1	100
Longer run	[1, 2, 3]	1	1,000
Higher noise	[1, 2, 3]	3	1,000

Compare estimated means, selection counts, and recommendations in a short README discussion. Explain the effects of more observations and higher noise, including any unexpected result.

Check that rerunning an identical configuration and seed reproduces the result, and that selection counts sum to the number of rounds.

6. Push to GitHub and deploy on Streamlit

Push your project to GitHub. Your README must include setup instructions, explanations of both models, experiment results, and project links.

Deploy on Streamlit Community Cloud: select your repository, branch **main**, entrypoint **main.py**, and Python 3.12. Verify that the deployed page runs. See the Streamlit deployment guide.

7. Present your application

Give a **3-5 minute presentation**: explain the Gaussian adaptation, demonstrate the deployed page, and discuss your experiments. Be ready to explain the posterior update and known σ. In a team, every member must contribute.

Grading

Criterion	Points
Independent frontend repository and local setup	10
Explanation of the original algorithm	15
Correct Gaussian Thompson sampling implementation	35
Working new page and clear results	20
GitHub repository and Streamlit deployment	10
Presentation and interpretation of experiments	10
Total	**100**

Submit: your GitHub repository URL and deployed Streamlit URL. Include your explanations and experiment results in the README, and be ready to give your presentation.